MARKHOR SYSTEMS

Internal. Not for the client.

DEVELOPER HANDBOOK

SoFlo Suds Customer App

Branded booking app on Urable, with rewards, reviews and admin

Item	Detail
Developer	Bilal Tahir
Prepared by	Ali Hassan, Co-founder
Communication	Project WhatsApp group (Bilal, Ali, Faizan)
Client timezone	US Eastern (EDT, UTC-4; EST, UTC-5 from 1 Nov). 9 hours behind Pakistan.
Milestone A deadline	Sunday 26 Oct 2026, 11:59 PM PKT: full working APK
Milestone B deadline	Monday 3 Nov 2026, 11:59 PM PKT: store-ready builds submitted
Version / date	v1.0, 20 Oct 2026

0. Read this first

This is everything you need to build the SoFlo Suds app. Read all of it before you write a line of code or open ChatGPT. If something is not in here, ask Ali in the project group before you build it. Do not guess, and do not build extra things.

The agenda, word for word

1. Today (Tue 21 Oct): read this handbook, open the client's concept app, test the Urable API with the key.

2. By Wed 22 Oct, 9:00 PM PKT: design PDF of every screen (app + admin) sent to Ali.

3. By Fri 24 Oct, 11:59 PM PKT: first APK with login and full booking into Urable working.

4. By Sun 26 Oct, 11:59 PM PKT: Milestone A, full APK with every feature in Section 1.3 working.

5. Mon 27 Oct to Sun 2 Nov: client feedback fixes, iOS build, store assets.

6. By Mon 3 Nov, 11:59 PM PKT: Milestone B, both store builds submitted and every item in Section 9 ticked.

7. Every day by 10:00 PM PKT: post progress in the group (what is done, what is next, what is blocked). A new APK every time a feature works.

Rules that do not bend

1. These deadlines are fixed. If you see you will miss one, tell Ali the same day, not on the deadline.
2. Build only what is in Section 1.3. Anything else the client asks for goes to Ali first. You never agree to new work with the client.
3. Never talk about price, payments or extra cost with the client or in the group. Send those questions to Ali privately.
4. The Urable API key never goes in the app, the repo, a screenshot or a chat message. Server only, from the password manager.
5. Push every change to the company repo every day. Keep the README updated.
6. Every milestone ends with something installable (APK), never a screen recording.
7. Leave or exams: tell Ali at least 3 days before, so a backup can be arranged.

1. What we are building

A branded customer app for SoFlo Suds, an auto detailing, ceramic coating and paint protection film (PPF) business. The shop already runs its jobs, bookings and payments in Urable (a shop management platform). Customers will book through our app, and every booking lands in the shop's Urable account. Features Urable does not have (rewards, referrals, promo codes, reviews, gallery, protection records, shop) run on our own server.

1.1 Core loop

```
Customer opens app
      |
      v
Sign up / Login  ---->  customer created in Urable (+ our users table)
      |
      v
Book: service -> vehicle -> in-shop or mobile -> date & time -> review & pay
```

```
       |
       v
Job created in Urable  ---->  push: "Booking confirmed"
       |
       v
Day before: push reminder      Status changes in Urable -> push to customer
       |
       v
Job completed + paid  ---->  points added, referral reward (if any)
       |
       v
Customer leaves review  ->  sees protection record  ->  rebooks in one tap
```

1.2 Roles

Role	What they can do
Guest	See splash and welcome screens only. Must sign up or log in to see services and book.
Customer	Book, pay, manage own vehicles, see own bookings, status, points, referral code, own protection records; review own completed jobs; browse gallery, reviews, shop. Never sees other customers' data.
Admin (shop owner)	Web admin panel only. Manage promo codes, loyalty rules and rewards, referral reward, gallery, reviews (hide/show), warranty details, shop products, send promo push. Cannot see card details (Urable holds payments).
Shop staff	Keep using the Urable app as today. No access to our app or admin.

1.3 Full feature list (build all of this, nothing more)

#	Feature	Notes
1	Branding	SoFlo Suds logo, colors, app icon, splash. Design follows the client's concept app (Section 1.6).
2	Auth	Email + password sign up, login, forgot password, logout.
3	Booking	Service, vehicle, in-shop or mobile (+ address), date and time, review, pay. Creates a job in Urable.
4	Online payment	Through the payment method connected in Urable. We never touch card numbers.
5	Job status	Status from Urable shown in My bookings, live.
6	Push notifications	Booking confirmed, day-before reminder, status changes, admin promo push.
7	Booking history + rebook	Upcoming and past jobs; cancel; rebook same service and vehicle in one tap.
8	My vehicles	Add, edit, delete vehicles (stored in Urable).
9	Promo codes	Applied at Review & pay. Created in admin.
10	Loyalty points	Earned on completed, paid jobs; redeemed for rewards.
11	Referrals	Personal code; reward when a referred customer completes a first paid job.
12	Reviews	Rate + review a completed job, one review per job.
13	Before/after gallery	Photos uploaded by admin.
14	Protection records	Per vehicle: protection jobs done (coating, PPF, tint, windshield film) + warranty details.

#	Feature	Notes
15	Shop	Products listed by the shop, added to a booking and paid with it. No shipping or delivery.
16	Admin panel (web)	See Section 5.3.
17	Profile	Edit details, payment history, notification settings, contact shop, privacy policy, logout.
18	Store submission	Google Play + Apple App Store, store listing, screenshots, privacy policy page.

1.4 What we are NOT building

- Memberships or monthly subscriptions
- Staff or technician app
- In-app chat
- Spanish or any second language
- Our own booking or payment system replacing Urable
- Shipping, delivery or stock management for shop products
- Anything in the client's concept app that is not in Section 1.3. Note it and send it to Ali.

1.5 Domain basics (read this so the app feels right)

- **Detailing:** deep cleaning of a car inside and out (interior, exterior, engine, headlights, water spots, pet hair).
- **Paint correction:** polishing out swirls and scratches before protection is applied.
- **Ceramic coating:** a hard protective layer on paint, glass, wheels, leather or fabric. Has a warranty in years. The shop uses Feynlab coatings.
- **PPF (paint protection film):** clear film on the paint against chips and scratches, gloss or matte. The shop uses XPEL film, which comes with a manufacturer warranty.
- **Window tint / windshield protection film:** film on glass.
- **Mobile vs in-shop:** mobile means the team drives to the customer's home or office; in-shop means the customer brings the car to the shop.
- **Vehicles:** cars, trucks, boats and RVs. Customers are often owners of premium cars (Porsche, Tesla, Corvette, BMW), so the app must look premium and calm, not flashy.

Why protection records matter: a coating or PPF job is expensive and has a warranty. The customer wants to see, per car, what was done, when, with which product, and until when it is covered. That is the "protection record".

1.6 What the client gave us

Reference	Status	How to use it
Client's concept app soflo-suds-app-demo.exactmyna33.chatgpt.site	Live, clickable demo. No real backend.	Design direction for the whole app. Walk every screen and button on Day 1, screenshot each one into the repo /docs/concept folder. Match its look; build only features in 1.3.
Client website soflosuds.com	Live	Source of brand: logo, colors, photos, service names and wording. Gallery seed photos can come from its gallery page.

Reference	Status	How to use it
Logo file soflosuds.com/wp-content/uploads/2026/08/Soflo-Suds-Auto_New_Logo-scaled.webp	Available	Logo for splash, icon, admin. Ask Ali if a higher-quality or transparent version is needed.
Urable online booking page app.urable.com/virtual-shop/v2utxe9P5ebXGz1m9fXr	Live	Shows the real services, prices and booking steps. Also our fallback for time slots or payment (Section 4.2).
Urable API docs app.urable.com/docs	Needs login with the client's account / key	Source of truth for the integration. Read on Day 1.
Urable API key	Old key exposed in chat. New key requested.	Password manager entry: "SoFlo Suds / Urable API key". Never paste it anywhere else. Use the current key for testing until the new one arrives.
Google Play + Apple developer accounts	Pending from client	Needed for Milestone B only. Ali is collecting them. Not a blocker for Milestone A.
"Plan & App Flow" document (sent 17 Oct)	Sent to client	Screen list and flow the client has seen. This handbook matches it.

What these references tell us

- The concept app is titled "SoFlo Suds | Your vehicle. Elevated." and has a dark theme (#08090C). Its sections are booking, rewards, protection records and shopping. The client designed it himself, so he will compare our screens to it.
- The website uses the tone "careful attention and clear communication" and certified, premium positioning. Keep app copy short, confident and clear.
- The shop offers both in-shop and mobile service. The shop address and the mobile service area come from config (ask Ali for exact values; do not hardcode).

The client's words (use them exactly in the UI)

Client says	We mean	Where it appears
Book Now / Schedule	Start booking flow	Home button
Protection records	Warranty + protection history per vehicle	Tab / screen 14
Rewards	Loyalty points + referral	Tab / screen 15
Customer portal	My bookings, vehicles, profile	Bottom tabs
Your vehicle. Elevated.	Tagline	Welcome screen
Ceramic Coating, Paint Protection Film, Window Tinting, Mobile Detailing, Paint Correction, Interior Detailing, Exterior Detailing	Service categories	Services screen (names come from Urable; keep his spelling)

His constraints, in his own words

- "I want my branding splash screen and maybe customer portal but pointing to their app basically white labeling their app" (the app must feel like SoFlo Suds' own app, never like Urable).
- "So I can see it get built" (he expects to install APKs as features land; send each working build to Ali).
- "I want every option" (he will ask for more. Anything not in 1.3 goes to Ali. Never say yes yourself).
- If his wording and this handbook ever conflict on a label or text, his wording wins. If they conflict on scope, ask Ali.

Week Zero: design and alignment (compressed: 21 to 22 Oct)

> **Why it is compressed**
>
> Normally Week Zero runs 7 days. Here it is 2 days because of the deadlines, and the client already gave us a concept app. So you are not inventing a design. You are cleaning his concept into a consistent, buildable system and covering every screen and state. No feature code in Week Zero, but you may set up the repo and test the Urable API.

The idea: settle every design question now, where it is cheap, so it does not become rework later. You own Week Zero. ChatGPT generates and assists; you decide, review and own the result. Never accept the first output.

W0.1 Process (in this order)

1. **Read the brief:** this handbook, end to end, until you can explain the product, users, features and flows in your own words.
2. **Discover the domain:** read Section 1.5, browse soflosuds.com (services, gallery, reviews pages) and the Urable booking page.
3. **Users and flows:** map the journeys in Section 1.1 and derive screens from them (not the other way round).
4. **Client design requirements:** walk the concept app fully; note logo, colors, type, layout patterns, and anything that looks inconsistent.
5. **Visual theme:** take the concept app's dark theme and the website brand. Define: colors (primary, accent, background, surface, text), semantic colors (success, warning, error, info), type scale, corner radius, shadows, icons, imagery, buttons, cards, navigation. Justify any change from the concept in one line.
6. **Screen inventory with states:** Section 5, every screen with empty, loading, error, success, disabled, logged-out and offline states. Screen count is never a target in itself.
7. **Prepare full ChatGPT context:** use the 12-area prompt in W0.3.
8. **Generate the UI.**
9. **Iterate on specific weaknesses:** name the exact problem, never "make it better".
10. **Internal review of the whole flow:** can a customer understand it, finish a booking and predict what happens next?
11. **Mockup PDF:** cover, design direction, palette, typography, UI principles, all screens, flow organisation, states, notes and open questions. It must be understandable without you explaining it.
12. **Submit to Ali** by Wed 22 Oct, 9:00 PM PKT.

W0.2 Branding guide (in this order)

1. **Brand colors** for the niche (from the concept + website).
2. **Logo:** given. Make the app icon (Android adaptive + iOS) and splash from it. Splash can be static, or a 2.5 to 3 second animated video made with Google's video model.
3. **Logo on socials:** profile image versions (square, round) for the shop's socials.
4. **All app UI screens** (Section 5.1) and all admin web screens (Section 5.3).
5. **Store screenshots:** Play Store and App Store sets. Can be finished during Milestone B, but plan them now.

W0.3 ChatGPT prompt structure (12 areas)

```
1. Role:            You are a senior mobile UI/UX designer for premium service apps.
2. Project context: Branded customer app for an auto detailing / ceramic coating / PPF
shop.
3. Domain:          (paste Section 1.5)
4. Users:           Customers (often premium car owners); admin uses a web panel.
5. Features:        (paste Section 1.3)
6. Flows:           (paste Section 1.1 core loop)
7. Screen inventory:(paste Section 5.1 with states)
8. Brand:           Logo + colors from the client's concept app and website; dark theme
#08090C.
9. Visual direction:Premium, calm, high-contrast, photo-led. Clean, consistent spacing.
10. Quality bar:    Coherent, realistic, usable. No neon, glow, random glassmorphism,
gradients.
11. Platform:       iOS + Android phones (React Native), 375-430 pt wide; admin web 1280+
px.
12. Output:         One screen at a time, with all its states, consistent with previous
screens.

Iteration example: "Home has three competing CTAs. Keep Book Now as the only primary
action;
keep the same colour and type system."
```

W0.4 Approval flow

```
Bilal -> self-review -> Ali (supervisor) -> revisions -> Client -> revisions -> approved
-> build
```

Two rounds of client design changes are planned. Ali sends the PDF to the client and brings back feedback. Do not wait idle for approval: from 23 Oct start building the screens the client is least likely to change (auth, backend, Urable adapter) while feedback comes in.

W0.5 Week Zero done when

- ☐ Product understood and explainable in your own words
- ☐ Concept app fully walked and screenshotted into /docs/concept
- ☐ Urable API tested with the key: list of what works and what does not (Section 4.1) posted to the group
- ☐ Visual direction set and written down
- ☐ Every screen in Section 5 designed with its states
- ☐ Mockup PDF complete and sent to Ali
- ☐ Open questions listed for Ali (Section 10)

2. Architecture



```
  +---------------------+                                  |              |
  |  Admin web (React)  | ---- HTTPS (admin JWT) ------->|              |
  +---------------------+                                  v              v
                                                      PostgreSQL      Urable API
                                                     (our data)    (customers, vehicles,
                                                                    services, jobs,
payments)
```

Layer	Choice	Why
Mobile	React Native with Expo (TypeScript), Expo Router	One codebase for iOS + Android; EAS builds APK and store builds fast.
State / data	TanStack Query + Zustand	Server cache with retries; small local state.
Backend	Node.js 20, Express or Fastify, TypeScript, Zod validation	Team stack; fast to build.
Database	PostgreSQL (managed: Supabase or Neon) with Prisma	Relational data (ledgers, codes); easy migrations.
Push	Expo Push (FCM + APNs)	One API for both platforms.
Files	Supabase Storage or S3-compatible bucket	Gallery photos.
Admin	React + Vite, served as a separate web app	Simple CRUD screens.
Hosting	Render / Railway (API + cron), managed Postgres	Ask Ali which account to use before deploying. Do not create paid accounts yourself.

Repo layout (one company repo, monorepo)

```
soflo-suds/
  apps/mobile/        Expo app
  apps/admin/         React admin panel
  apps/api/           Node API (routes, services, urable/, jobs/)
  packages/shared/    shared types, Zod schemas, constants
  docs/concept/       screenshots of the client's concept app
  docs/urable/        API test notes and sample responses (no keys)
  README.md           setup, env vars, build and release steps (keep updated)
```

Non-negotiable rules

- The Urable API key lives only in the API server env. The app never calls Urable directly.
- All Urable calls go through one adapter (apps/api/src/urable/). If Urable changes or a feature needs the fallback, only the adapter changes.
- The server decides everything that has value: prices, discounts, points, referral rewards, review eligibility. The app only displays.
- Points are an append-only ledger. Never edit or delete a ledger row; correct with a new row.
- Every reward action is idempotent (keyed on the Urable job id), so a retry or a double webhook never pays twice.
- No card data on our server, ever. Payment happens through Urable.
- All numbers the client did not set (points per dollar, reward costs, reminder time) are config in the settings table, not constants.

3. Data model (our PostgreSQL)

Urable is the source of truth for customers, vehicles, services, jobs and payments. Our database keeps the link to Urable plus everything Urable does not have.

users

Column	Type	Notes
id	uuid PK	
urable_customer_id	text UNIQUE	Set on sign-up after creating the customer in Urable
email	citext UNIQUE	Lowercased
password_hash	text	bcrypt, cost 12
first_name, last_name	text	
phone	text	E.164, unique when present
referral_code	text UNIQUE	6 chars, A-Z 2-9, no lookalikes
referred_by_user_id	uuid FK users NULL	Set once at sign-up; never changed
notif_prefs	jsonb	{booking, reminders, status, promos}: all true by default
created_at, deleted_at	timestamptz	Soft delete for account deletion (store requirement)

push_tokens

Column	Type	Notes
id	uuid PK	
user_id	uuid FK	
token	text UNIQUE	Expo push token
platform	text	ios | android
updated_at	timestamptz	Remove tokens Expo reports as invalid

job_sync

Column	Type	Notes
urable_job_id	text PK	
user_id	uuid FK	
last_status	text	Last status seen from Urable
scheduled_at	timestamptz	For the day-before reminder
reminder_sent_at	timestamptz NULL	Idempotency for reminders
completed_paid_at	timestamptz NULL	Set once when the job is completed AND paid; triggers points
promo_redemption_id	uuid NULL	If a code was used

settings

Column	Type	Notes
key	text PK	e.g. points_per_usd, referral_reward_points, reminder_hour_local, shop_address, mobile_area_text, shop_phone, shop_timezone
value	jsonb	Edited from admin

promo_codes

Column	Type	Notes
id	uuid PK	
code	citext UNIQUE	Uppercase
type	text	percent | fixed
value	numeric(10,2)	Percent 1-100 or USD
min_order_usd	numeric NULL	
starts_at, ends_at	timestamptz	
max_total_uses	int NULL	
max_uses_per_user	int	Default 1
first_booking_only	bool	
active	bool	

promo_redemptions

Column	Type	Notes
id	uuid PK	
promo_code_id	uuid FK	
user_id	uuid FK	
urable_job_id	text UNIQUE NULL	Linked once the job is created
discount_usd	numeric	Computed by server
status	text	reserved | applied | released

points_ledger (append-only)

Column	Type	Notes
id	bigserial PK	
user_id	uuid FK	
delta	int	+ earn, - redeem, +/- adjust
reason	text	job_completed | referral_referrer | referral_referee | redeem | admin_adjust
ref_id	text	urable_job_id or redemption id
created_at	timestamptz	
UNIQUE	(reason, ref_id, user_id)	Makes every award idempotent

rewards / reward_redemptions

Column	Type	Notes
rewards.id, title, description, points_cost, active, image_url	various	Created in admin (e.g. "Free interior refresh")
reward_redemptions.id, user_id, reward_id, status, code, created_at	various	status: issued | used | cancelled. code shown to staff at the shop

referrals

Column	Type	Notes
id	uuid PK	
referrer_id, referee_id	uuid FK	referee_id UNIQUE (one referrer per customer)
status	text	pending | rewarded | rejected
reject_reason	text NULL	self_referral | duplicate_device | duplicate_phone

reviews

Column	Type	Notes
id	uuid PK	
user_id	uuid FK	
urable_job_id	text UNIQUE	One review per job
rating	smallint	1-5
text	text	Max 1,000 chars
hidden	bool	Admin can hide
created_at	timestamptz	

gallery_items

Column	Type	Notes
id	uuid PK	
before_url, after_url	text	after_url required, before optional
caption, service_tag	text	
sort_order	int	
published	bool	

warranties

Column	Type	Notes
id	uuid PK	
urable_vehicle_id	text	
urable_job_id	text NULL	
protection_type	text	ceramic_coating | ppf | tint | windshield_film | other

Column	Type	Notes
product	text	e.g. "Feynlab Ceramic Lite", "XPEL Ultimate Plus"
coverage	text	e.g. "Full front", "Full body"
applied_on, expires_on	date	
notes	text	

shop_products

Column	Type	Notes
id	uuid PK	Only if Urable products are not available via API
urable_item_id	text NULL	Preferred: link to a Urable product
name, description, image_url	text	
price_usd	numeric	Display only; Urable charges the real price
active	bool	

admin_users / notifications_log

Column	Type	Notes
admin_users: id, email, password_hash, created_at	various	Seed one admin; no public sign-up
notifications_log: id, user_id, type, title, body, sent_at, ref_id	various	UNIQUE (type, ref_id, user_id) to stop duplicate pushes

4. Mechanics in detail

4.1 Urable adapter (Day 1 task)

Urable's API is new and we have not seen its docs. On Day 1, log in to the docs with the key and test each function below. Post the result table in the group and save sample responses (with keys removed) in /docs/urable.

Adapter function	What we need	If Urable does not support it
createCustomer / getCustomer	Create on sign-up, read profile	Blocker: tell Ali the same day
listVehicles / createVehicle / updateVehicle / deleteVehicle	Customer's vehicles	Store vehicles in our DB and send vehicle details in the job notes
listServices (items, categories, prices, add-ons, duration)	Services screen	Blocker: tell Ali
getAvailability(date, service, location)	Free time slots	Fallback A (below)
createJob (customer, vehicle, services, time, address, notes)	Booking	Fallback A
getJob / listJobs(customer)	My bookings, status, history	Blocker: tell Ali
payment link / invoice / checkout for a job	Review & pay	Fallback B
discount line on job/invoice	Promo codes, reward redemption	Fallback C

Adapter function	What we need	If Urable does not support it
listProducts	Shop	Use shop_products table, add products as job line items or notes
webhooks (job created/updated/completed, payment)	Status push	Polling (Section 4.3)

```
Fallback A (no slots / no job creation via API):
  Date & time + confirm step opens the Urable virtual shop booking page in an in-app
  browser (WebView), pre-filled where possible, styled header in SoFlo colors.
  After the customer finishes, we read the new job via listJobs and link it.

Fallback B (no payment via API):
  Open the Urable invoice / payment page for the job in the in-app browser.
  Mark "paid" only when Urable reports it.

Fallback C (no discounts via API):
  Add the promo / reward as a job note ("PROMO FIRST10: -10%") and a line in the
  booking summary. Flag it to Ali so the shop applies it in Urable.
```

4.2 Booking flow

```
1. GET /services                -> from Urable (cache 10 min)
2. pick service (+ add-ons)
3. pick / add vehicle           -> Urable vehicle
4. pick location: in-shop | mobile (address required, validate not empty)
5. GET /availability?date&serviceId&location
6. POST /bookings/quote  {serviceIds, addOnIds, productIds, promoCode?,
rewardRedemptionId?}
     server: price = sum(Urable prices); discount = applyPromo(); total = max(0, price -
discount)
     returns {lines[], discount, total, quoteId (valid 15 min)}
7. POST /bookings {quoteId, vehicleId, slot, location, address?}
     server: re-check slot + promo; reserve promo; Urable.createJob(...)
     store job_sync row; push "Booking confirmed"
8. Pay: Urable payment (or Fallback B). App shows status from server.
```

Edge cases: slot taken between steps 5 and 7: return SLOT_TAKEN and show the next 3 free slots. Urable down: return URABLE_UNAVAILABLE, keep the customer's choices, show "Try again". Double tap on Confirm: the client sends an Idempotency-Key header; the server returns the same booking.

4.3 Status sync, reminders, notifications

```
Every 5 min (cron):  for each job_sync where status not in (completed, cancelled)
                       and scheduled_at within -2 days .. +30 days:
                         s = Urable.getJob(id).status
                         if s != last_status: push "status_<s>" (once,
notifications_log); update
                         if s == completed AND paid AND completed_paid_at is null:
                             set completed_paid_at; awardPoints(job);
settleReferral(user)
Every 15 min (cron): jobs with scheduled_at between now+20h and now+28h, reminder_sent_at
null:
```

```
                          push reminder at settings.reminder_hour_local (shop timezone);
set reminder_sent_at
If Urable has webhooks: use them for instant status, keep the cron as a safety net.
```

Respect notif_prefs. Promo pushes from admin go only to users with promos = true, max 1 per day.

4.4 Promo codes

```
applyPromo(code, user, subtotal):
  p = promo_codes where code = upper(code) and active and now between starts_at and
ends_at
  reject if not found                          -> PROMO_INVALID
  reject if p.min_order_usd > subtotal         -> PROMO_MIN_ORDER
  reject if p.first_booking_only and user has any job -> PROMO_FIRST_BOOKING_ONLY
  reject if user's applied+reserved uses >= p.max_uses_per_user -> PROMO_ALREADY_USED
  reject if total uses >= p.max_total_uses     -> PROMO_LIMIT_REACHED
  discount = percent ? round(subtotal * value / 100, 2) : min(value, subtotal)
Reserved on booking; applied when the job is paid; released if cancelled.
```

Abuse defence: codes are validated only on the server; limit checks run inside a transaction with a row lock on the promo; 10 wrong codes per user per hour then PROMO_RATE_LIMITED.

4.5 Loyalty points (numbers are OUR PROPOSAL, stored as settings)

```
earn on completed + paid job:  points = floor(amount_paid_usd * settings.points_per_usd)
// proposal: 1
insert points_ledger (user, +points, 'job_completed', urable_job_id)  -- UNIQUE stops
double award
balance = SUM(delta) for user   (never stored as a single editable number)
redeem reward: in one transaction, lock user, check balance >= points_cost,
               insert ledger (-points_cost, 'redeem', redemption_id), create
reward_redemption
```

Edge cases: amount_paid is the amount after discounts, excluding tips. Job refunded or cancelled after award: insert a negative admin_adjust row. Balance can never go below 0 on redeem.

4.6 Referrals

```
sign-up with code -> referrals(pending) if code valid and not own code
on referee's FIRST completed + paid job:
  reject if same phone, same device id or same payment name as referrer -> rejected
  else: ledger +settings.referral_reward_points to BOTH (reasons referral_referrer /
referral_referee,
        ref_id = referral.id) -> status rewarded; push both
```

Abuse defence: reward only after a real paid job, never at sign-up; one referrer per customer; max 20 rewarded referrals per referrer per month (setting).

4.7 Reviews

- Only for the customer's own jobs with status completed. One per job (UNIQUE). Prompt with a push 2 hours after completion.
- Rating 1-5 required; text optional, 1,000 chars max, stripped of HTML. Admin can hide, not edit.

4.8 Protection records

- Build the list per vehicle from the customer's completed Urable jobs whose service category is ceramic coating, PPF, tint or windshield film (map categories in settings).
- Join warranty rows the admin entered (product, coverage, applied_on, expires_on). Show "Covered until <date>" or "Warranty details coming soon" if none yet.
- Read-only for customers. Sort newest first.

4.9 Shop

- Products from Urable if the API lists them, otherwise shop_products. Customer adds products in the booking flow (step 2) or from the Shop screen ("Add to next booking").
- Products are charged with the booking through Urable. No cart checkout of its own, no shipping.

5. Screen inventory

Every screen needs these states unless marked N/A: loading (skeleton), empty, error with retry, offline banner, success. Logged-out users are sent to Welcome.

5.1 App screens

#	Screen	Purpose and content	States to build
1	Splash	Logo (static or 2.5-3 s video)	First launch, returning user
2	Welcome	Tagline "Your vehicle. Elevated.", 2-3 slides: certified team, protection, rewards; Sign up / Log in	N/A
3	Sign up / Login / Forgot	Name, email, phone, password, referral code (optional)	Field errors, email taken, wrong password, invalid referral code, reset email sent
4	Home	Greeting, next booking card, Book Now (only primary CTA), promo banner, quick links (Rewards, Protection records, Gallery)	No bookings yet, upcoming booking, booking in progress
5	Services	Categories and services from Urable with price and duration	Empty, Urable down
6	Service details	Photos, what is included, duration, price, add-ons, products to add	Add-ons selected
7	Choose vehicle	Saved vehicles + add new (make, model, year, color, plate optional)	No vehicles, validation errors
8	Location	In-shop (shop address from settings) or Mobile (address input)	Address missing
9	Date & time	Calendar + slots (or Fallback A web view)	No slots that day, slot taken
10	Review & pay	Lines, promo code field, reward to use, total, Pay	Promo valid/invalid, payment pending, payment failed
11	Booking confirmed	Summary, add to calendar, go to My bookings	N/A
12	My bookings	Upcoming / Past tabs; status chip; cancel; rebook; review prompt on completed	Empty, cancel confirm, cancel not allowed
13	My vehicles	List, add, edit, delete	Empty, delete confirm, vehicle has upcoming job

#	Screen	Purpose and content	States to build
14	Protection records	Per vehicle: protection jobs, product, coverage, applied, covered until	No protection yet, warranty pending, expired
15	Rewards	Points balance, history, rewards to redeem, referral code + share	Not enough points, redeemed (show code), no rewards configured
16	Gallery & reviews	Before/after slider grid; reviews list with average; write review	No photos, no reviews, already reviewed
17	Shop	Products grid, details, "Add to next booking"	Empty, no upcoming booking
18	Profile	Edit details, payment history (from Urable), notification settings, contact shop (call/email from settings), privacy policy, delete account, logout	Delete confirm

Navigation: bottom tabs: Home, Bookings, Rewards, Protection, Profile. Gallery and Shop open from Home quick links. Push taps deep-link to the right booking.

5.2 System screens

- No internet, forced update (min version from settings), maintenance message, push permission explainer (shown before the OS prompt).

5.3 Admin web screens

Screen	Content
Login	Email + password
Dashboard	Counts: users, bookings this week, points issued, active promos
Promo codes	List, create, edit, deactivate; usage count
Loyalty & rewards	Points per $ setting, rewards list (create/edit/deactivate), redemptions with code and mark as used
Referrals	Reward points setting, list with status and reject reason
Reviews	List, hide/show
Gallery	Upload before/after, caption, tag, order, publish
Protection records	Search customer -> vehicle -> add/edit warranty
Shop products	Only if not from Urable: create/edit/deactivate
Push	Send promo push (title, body, optional deep link), history
Settings	Shop address, phone, email, mobile area text, reminder hour, min app version, category mapping

6. API surface

Method	Path	Auth	Notes
POST	`/auth/signup`	public	Creates Urable customer + user
POST	`/auth/login`	public	Rate limited
POST	`/auth/refresh`	refresh	
POST	`/auth/forgot, /auth/reset`	public	Token by email, 30 min

Method	Path	Auth	Notes
GET/ PATCH/ DELETE	/me	customer	DELETE = account deletion
PUT	/me/push-token	customer	
GET	/services, /services/:id	customer	From Urable, cached
GET/ POST/ PATCH/ DELETE	/vehicles[/:id]	customer	Own only
GET	/availability	customer	
POST	/bookings/quote	customer	
POST	/bookings	customer	Idempotency-Key header
GET	/bookings, /bookings/:id	customer	Own only
POST	/bookings/:id/cancel	customer	
GET	/bookings/:id/pay	customer	Returns Urable payment URL or status
GET	/rewards, /me/points	customer	
POST	/rewards/:id/redeem	customer	
GET	/me/referral	customer	Code + stats
GET/POST	/reviews, /bookings/:id/review	customer	
GET	/gallery	customer	
GET	/protection?vehicleId	customer	
GET	/shop/products	customer	
GET	/config	public	Shop info, min version
*	/admin/*	admin	All admin CRUD + push

Error envelope

```
{ "error": { "code": "SLOT_TAKEN", "message": "That time was just booked.", "details": {...} } }
```

Code	App behaviour
VALIDATION_FAILED	Show field errors
UNAUTHORIZED	Refresh token, else go to Login
EMAIL_TAKEN	Offer login
REFERRAL_INVALID	Clear field, continue sign-up
SLOT_TAKEN	Show next 3 free slots
URABLE_UNAVAILABLE	Keep choices, "Try again"
PROMO_INVALID / PROMO_MIN_ORDER / PROMO_ALREADY_USED / PROMO_LIMIT_REACHED / PROMO_FIRST_BOOKING_ONLY / PROMO_RATE_LIMITED	Message under the code field

Code	App behaviour
INSUFFICIENT_POINTS	Disable redeem, show points needed
REVIEW_NOT_ALLOWED / REVIEW_EXISTS	Hide write review
CANCEL_NOT_ALLOWED	Show "Call the shop" with shop phone
NOT_FOUND / FORBIDDEN	Generic error state

7. Security and abuse

- HTTPS only. JWT access token 15 min + refresh token 30 days (rotated, stored in SecureStore on device).
- bcrypt passwords; login and forgot-password rate limits (5 per 15 min per IP + email).
- Every customer route checks ownership (the job, vehicle or review belongs to the user's urable_customer_id).
- Zod validation on every input; strip HTML from text.
- Secrets only in env vars; .env never committed; Urable key from the password manager.
- Admin panel: separate admin JWT, strong password, no public sign-up.
- Logs never contain the API key, passwords, tokens or full addresses.
- Defences per mechanic are written next to each mechanic in Section 4 (promos, points, referrals, reviews).
- Account deletion inside the app (Apple and Google require it): soft delete, anonymise, revoke tokens.

8. Environments and release

Env vars (API)

```
DATABASE_URL=
URABLE_API_BASE=
URABLE_API_KEY=            # from password manager only
JWT_SECRET=  JWT_REFRESH_SECRET=
EXPO_ACCESS_TOKEN=         # push
STORAGE_BUCKET=  STORAGE_KEY=  STORAGE_SECRET=
SMTP_HOST= SMTP_USER= SMTP_PASS=  MAIL_FROM=    # password reset
ADMIN_SEED_EMAIL=  ADMIN_SEED_PASSWORD=
APP_ENV=dev|staging|prod
```

Env vars (mobile / admin)

```
EXPO_PUBLIC_API_URL=      VITE_API_URL=
```

Environment	Use
dev	Your machine. Urable test data only (create test customers clearly named "TEST").
staging	Builds sent to Ali and the client (APKs). Same Urable account; test bookings must be cancelled the same day and marked TEST.
prod	Store builds. Created for Milestone B.

> **Careful: Urable is the client's live business account**

Environment	Use

> Any test booking you create shows up in the shop's real calendar. Always name test customers "TEST Markhor", book far-future dates, put "TEST - please ignore" in notes, and cancel them the same day. Never charge a real card.

Build and submission path

```
APK (staging):   eas build -p android --profile preview   -> share link in group
iOS internal:    eas build -p ios --profile preview        (needs Apple account,
Milestone B)
Store builds:    eas build --profile production  ->  eas submit -p android / -p ios
App id proposal: com.soflosuds.app  (confirm with Ali before first production build)
```

Store checklist (Milestone B)

- ☐ Developer access to the client's Google Play and Apple accounts (Ali provides)
- ☐ App name, short + full description, keywords, category (Auto & Vehicles / Lifestyle)
- ☐ Screenshots: Play (phone) and App Store (6.7" and 6.5")
- ☐ Feature graphic (Play), app icon 1024 px
- ☐ Privacy policy page hosted (URL), data safety form (Play), privacy nutrition labels (Apple)
- ☐ Account deletion in app + URL
- ☐ Demo login for Apple review (TEST account) and review notes explaining booking is via the shop's system
- ☐ Content rating questionnaire
- ☐ Make sure the app has its own value (rewards, records, gallery) so Apple does not reject it as a website wrapper (Guideline 4.2)

9. Schedule and definition of done

All times are Pakistan time. Post progress in the group every day by 10:00 PM. Each step ends with something installable.

Date	Work	Must be in the group by end of day
Tue 21 Oct	Week Zero: read handbook, walk concept app, test Urable API, set repo	Urable test table (4.1), concept screenshots, repo link
Wed 22 Oct	Week Zero: theme, all app + admin screens, mockup PDF	Mockup PDF by 9:00 PM
Thu 23 Oct	API skeleton, DB + migrations, auth, Urable adapter; app shell, theme, navigation, auth screens	Login works against our API
Fri 24 Oct	Services, vehicles, location, date & time, quote, booking into Urable, payment; push setup	APK 1: sign up -> real booking in Urable -> push
Sat 25 Oct	My bookings, status sync cron, reminders, cancel, rebook, profile, payment history; promo codes; points ledger + rewards; referrals	APK 2

Date	Work	Must be in the group by end of day
Sun 26 Oct	Reviews, gallery, protection records, shop, admin panel (all screens), design fixes from client feedback	MILESTONE A: full APK + admin URL, every box below ticked
27 Oct - 1 Nov	Client feedback fixes (sent by Ali), polish, all states, iOS build, store assets, privacy policy, prod setup	Fix list closed daily
Sun 2 Nov	Full test pass on real Android + iPhone, production builds	Production builds ready
Mon 3 Nov	Submit to Google Play and App Store	MILESTONE B: both submitted

9.1 Done checklist per module (Milestone A)

Branding

- ☐ Icon and splash on both platforms
- ☐ Colors, type, buttons consistent on every screen
- ☐ Matches the approved design

Auth

- ☐ Sign up creates Urable customer
- ☐ Login, logout, forgot/reset work
- ☐ Referral code accepted at sign-up

Booking

- ☐ Full flow creates a real job in Urable (TEST)
- ☐ In-shop and mobile both work
- ☐ Slot taken and Urable-down handled
- ☐ Double tap does not double book

Payment

- ☐ Customer can pay through Urable (or Fallback B)
- ☐ Paid status shown from Urable

Bookings

- ☐ Upcoming/past correct
- ☐ Status updates within 5 min
- ☐ Cancel and rebook work

Push

- ☐ Confirmed, reminder, status, promo pushes arrive
- ☐ Settings toggles respected
- ☐ Tap opens the right screen

Vehicles

- ☐ Add/edit/delete synced with Urable

Promo codes

- ☐ All rules in 4.4 enforced on server
- ☐ Admin can create/deactivate

Points + rewards

- ☐ Points awarded once per paid job
- ☐ Redeem works, balance never negative

Referrals

- ☐ Both rewarded after first paid job
- ☐ Self-referral rejected

Reviews

- ☐ Only completed jobs, one per job
- ☐ Admin can hide

Gallery

- ☐ Admin upload shows in app

Protection records

- ☐ Protection jobs listed per vehicle
- ☐ Warranty from admin shown with covered-until date

Shop

- ☐ Products shown, added to booking, charged with it

Admin

- ☐ All screens in 5.3 work
- ☐ Settings change app behaviour without a new build

Profile

- ☐ Edit details, payment history, notifications, contact, privacy, delete account, logout

Quality

- ☐ No crash in a 30-minute test session
- ☐ All empty, loading, error, offline states
- ☐ README updated
- ☐ All test bookings cancelled in Urable

10. Open questions

Split every open item three ways: what is in our hands is done properly now, without waiting; what needs the client's choice goes to Ali to ask the client; external dependencies (accounts, keys) go on the pending list and never block the build.

Question	Blocks	Build against (working assumption)
What does the Urable API support? (slots, jobs, payments, discounts, products, webhooks)	Booking, payment, promo, shop	Adapter + fallbacks in 4.1. Answer on Day 1.
New Urable API key	Nothing yet	Use current key for dev/staging until the new one arrives
Deposit or full payment at booking?	Review & pay	Full payment through Urable
Cancellation rule	Cancel button	Free cancel up to 24 h before; after that "Call the shop"

Question	Blocks	Build against (working assumption)
Mobile service area and any mobile fee	Location step	Any address accepted; area text from settings; fee from Urable if Urable adds it
Points rules and rewards	Rewards content	OUR PROPOSAL: 1 point per $1; rewards created by admin; referral 250 points each. All settings.
Shop products: in Urable or admin?	Shop data source	Urable if API lists them, else admin
Warranty data for past customers	Protection records content	Empty state until admin enters them
Hosting accounts (API, DB, storage)	Staging deploy	Ask Ali before creating; dev runs locally
Google Play + Apple developer access	Milestone B only	Ali collects from client; not a blocker before 27 Oct
Parts of the concept app not in 1.3	Nothing	Do not build; list them for Ali on Day 1

Commercial terms, fees and payment are in the SOW and are not part of this handbook. Questions about them go to Ali privately.